

Report - Project 5 Restaurant

Project description

- bootstrap.sh

The bootstrap script acts as a control system before starting the simulation.

Its main functions are:

- Parameter management: it allows loading an environment file different from the default one and allows dynamic modification of .env variables.
- Data validation: it verifies the correct format of the .env file and the validity of the data types.
- Structural validation of CSV files: it analyzes the structure of the menu and resources files line by line.
- Compilation and execution: if the checks are successful, it compiles the C code by linking the pthread library and runs the application.

- status.sh

This is a simple Bash script that reads the restaurant's PID. After ensuring that it is a valid integer, it sends a SIGUSR1 signal to the process to force the printing of a status report.

- main.c

Represents the heart of the project. The main function performs the following operations:

- Writes its own PID in the file /tmp/restaurant.pid to communicate with status.sh.
- Initializes shared data structures and synchronization tools.
- Handles the arrival of the SIGUSR1 signal.
- Creates the threads and implements the simulation loop, scaled according to the game speed.

- customer.c

Represents the behavior of customers. Each customer thread performs the following operations:

- Initializes its own order, generating between MIN_DISHES and MAX_DISHES random dishes, and calculates the initial patience based on the total estimated cooking time and a bonus between MIN_BONUS_PATIENCE and MAX_BONUS_PATIENCE.
- Handles an active waiting loop where the remaining patience is progressively decremented, scaled according to the game speed.
- Determines its own exit from the restaurant upon receiving all dishes or due to patience expiration, updating the global score.

- cook.c

Represents the behavior of cooks. Each cook thread performs the following operations:

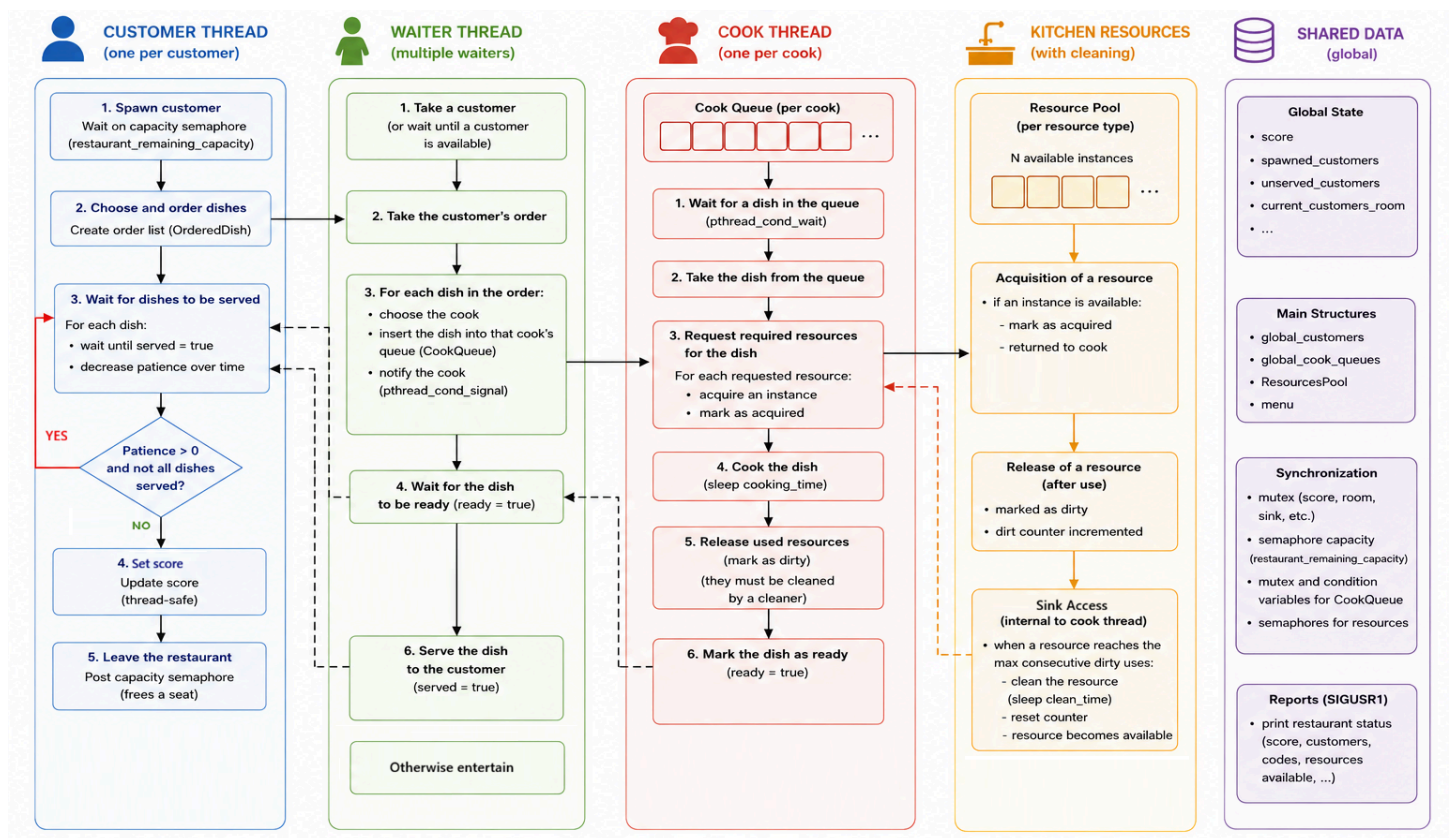
- Wait for a dish to be cooked.
- Once the order arrives, it checks which resources it needs and then tries to acquire them.
- Every time it also decides whether to stay idle, cook another dish or try to clean if the resources are not available.
- A cook always tries to clean a resource but if the sink is busy, it aborts that action and proceeds to use it dirty or just left it in there.

- waiter.c

Act as the coordination, dispatching, and scheduling link between the room and the kitchen. Waiter threads execute three main tasks:

- Assignment: scans active customer orders and distributes pending dishes into the cooks' queues based on load-balancing logic.
- Delivery: checks if dishes in the kitchen are ready, serves them to the respective customers, and updates the status.
- Entertainment: during idle times, waiters randomly entertain a customer, modifying their remaining patience positively or negatively.

Project Flowchart



Design choices

- Maximum Capacity Management

A semaphore is used to limit customer access to the restaurant based on the MAX_CUSTOMERS limit. In the main loop, using sem_trywait() instead of sem_wait() is necessary to prevent the program from blocking, allowing it to continue printing periodic reports and managing time even when the restaurant is full.

- Thread-Safety of Score

Access to the global score variable is always protected using a mutex.

- Use of Environment Variables and getenv()

Choosing to use environment variables to configure simulation parameters is useful to avoid re-parsing the .env file (which is already handled in bootstrap.sh) or passing a long list of arguments to the main function.

- Time Measurement via clock_gettime()

The clock_gettime system call was used to measure time, allowing for accurate calculation of time intervals.

- Use of pthread_detach()

Customer threads are generated dynamically and can terminate at any time, meaning that manual waiting via “pthread_join()” cannot be used.

This choice serves to avoid program blocks caused by the join operation or the creation of zombie threads.

- Customer waiting loop (polling)

At the end of the simulation the main thread must wait for all customers currently inside the restaurant and then can safely terminate other threads. The program cannot use “pthread_join()” to wait for customers because they are detached using “pthread_detach()”, so it uses a polling loop that checks the value of capacity semaphore.

- Load balancing and scheduling strategy

To assign dishes to the kitchen, the waiter inspects the queues of all cooks while protecting access via their respective mutexes, identifying the cook with the fewest pending dishes to ensure fair load balancing. For high-value orders or customers with critically low patience, the waiter automatically, distributes dishes from the same customer across multiple cooks to leverage parallel execution and maximize the probability of a positive score contribution. Otherwise, the order is assigned to a single cook.

- Multi-level synchronization of customer state

Each customer’s state is constantly exposed to concurrent modifications: it is decremented by the customer thread itself, altered by the waiter, and read by the cook and the waiter. To prevent race conditions a granular mutex has been implemented inside each individual customer structure.

- Deterministic Randomness

To ensure that the simulation remains deterministic, the project implements the xoshiro256++ algorithm for pseudo-random number generation, initialized via SplitMix64. The main thread initializes a master RNG generator based on RANDOM_SEED. Each cook, waiter and customer is assigned its own unique generator instance, derived from the master seed.

- Ordered Resources

The resources are ordered by quantity and the cook must follow that sequence when acquiring them in order to prevent deadlock. We implemented a bubblesort algorithm. This adds an $O(n^2)$ complexity to the code, which is fine in this case since the resources are limited in number.

- Error handling

A set of numeric error codes are defined in error_codes.h (C) and as shell variables (Bash).

- ERR_FILE_NOT_FOUND (1) — a required file does not exist.
- ERR_INVALID_PARSING (2) — a CSV or .env file is malformed.
- ERR_ENV_MISSING (3) — a required environment variable is absent.
- ERR_ALLOC_FAILED (4) — a malloc/calloc call returned NULL.
- ERR_SYS_FAILURE (5) — a system call (pthread, semaphore, open, write, ...) failed.
- ERR_NOT_READABLE (6) — a file exists but is not readable.
- ERR_NOT_VALID_VALUE (7) — a parameter has an out-of-range or wrong-type value.
- ERR_PROCESS_NOT_EXIST (8) — the PID stored in /tmp/restaurant.pid no longer refers to a running process.